13. Events in React JS

▼ What are Events in React?

- Events in React are the actions triggered like clicking a button, typing into a form field, or submitting a form.
- The React event system is based on the `SyntheticEvent` object, which is a cross-browser wrapper around the browser's native event.
- This ensures consistent behaviour across different browsers.

Examples of user actions:

- Clicking a button 👉 `onClick`
- Typing in an input box 👉 `onChange`
- Submitting a form 👉 `onSubmit`
- Moving the mouse 👉 `onMouseOver`

▼ Important Points:

1. React events are named using **camelcase** (e.g. `onClick`, `onChange`)
2. Event handlers are functions that execute when an event occurs.
3. The event object (`e - Synthetic Base Event`) provides the information about the event.
4. The `target` property in the event object refers to the element that triggered the event.

   Syntax – e.target

▼ What is a `Synthetic Event` in React?

- In React, a **Synthetic Event** is a **wrapper around the browser's native event** system.
- A **Synthetic Event** is React's **own version of the browser event**.
  - Works the **same in all browsers**

    - Gives **better performance**
    - Behaves like a normal JavaScript event
- In Simple Words:

> 💡 React converts all browser events into Synthetic Events for consistency.

- **Code**:

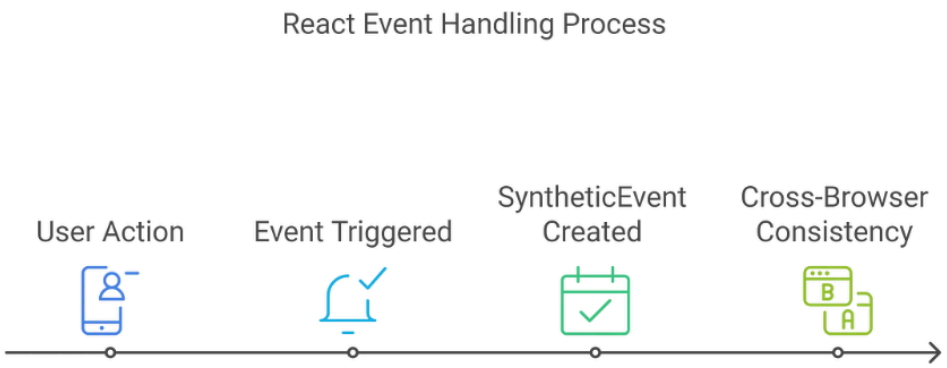
```
import React from "react";

const Event1 = () => {
  let handleClick = (event) => {
    console.log(event); // Synthetic Event in React
    console.log(event.nativeEvent); // Native event for the browser
    console.log("Button Clicked");
  };
  return (
    <>
      <button onClick={handleClick}>Click Me</button>
    </>
  );
};

export default Event1;
```

▼ Here's a list of common events in React:

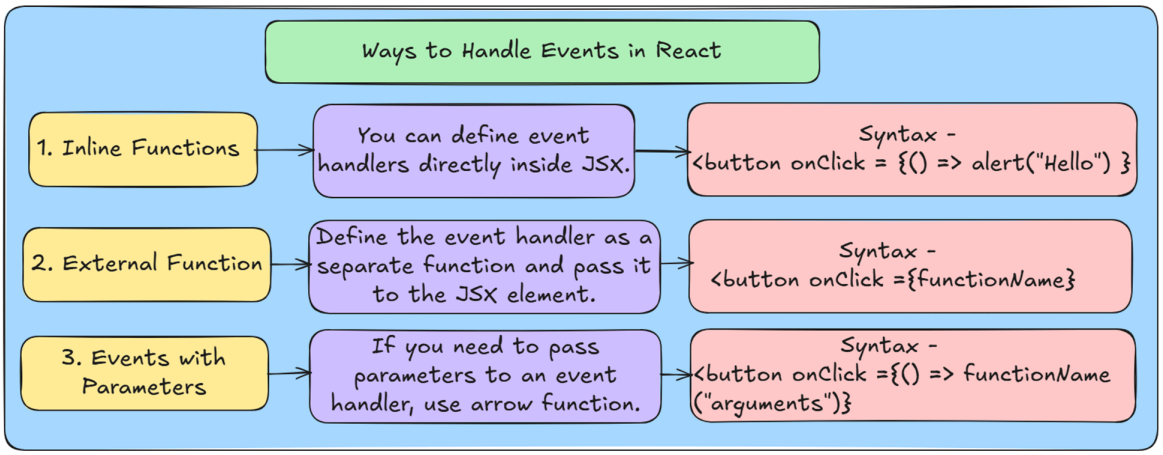
- `onClick`: Triggers when an element is clicked.
- `onChange`: Triggers when the value of an input field changes.
- `onSubmit`: Triggers when a form is submitted.



▼ Events Handling in React

- In react, event handling is similar to handing events in javascript but with some key differences.
- React uses a **SyntheticEvent**, a wrapper around native events for cross-browser consistency.

- **Ways to handle events in React:**



▼ `e.preventDefault()` in React

- In **React**, `e.preventDefault()` is used to **prevent the default behavior** of an event.
- When submitting a form, the default behavior is for the page to reload.
- `preventDefault()` stops this from happening, allowing React to handle the form logic instead.
- Technical Explanation:
  - `e` stands for the **SyntheticEvent** object in React.
  - `.preventDefault()` is a method on the event object that **stops the browser's default action** for that event.
- **Syntax:**

💡 **event.preventDefault();**

▼ 1. Inline Functions

- You can define event handlers directly inside JSX.
- **Code:**

```
import React from "react";

const InlineEvent = () => {
  return (
    <>
      <h1>Inline Event</h1>
      <button onClick={() => alert("Button Clicked")}>Click Me</button>
    </>
  );
};

export default InlineEvent;
```

▼ 2. External Functions

- Define a function separately and reference it in the event handler.

- **Code:**

```
import React from "react";

const ExternalFunctionEvent = () => {
  let handleClick = () => {
    alert("Button clicked by using external function");
  };
  return (
    <div>
      <h1>External Function Event</h1>
      <button onClick={handleClick}>Click Me</button>
    </div>
  );
};

export default ExternalFunctionEvent;
```

▼ 3. Events with Parameters

- You might want to pass **custom parameters** to the event handler - in that case, use an inline function wrapper.
- **Code:**

```
import React from "react";

const EventWithParameter = () => {
  let handleClick = (e, name) => {
    console.log(`Hello! ${name}`);
  };
  return (
    <div>
      <h1>Event With Parameter</h1>
      <button onClick={(e) => handleClick(e, "John")}>Get Name</button>
    </div>
  );
};
```

```
export default EventWithParameter;
```

- Why Not Call `handleClick("John")` Directly?
- If you write:

```
<button onClick={handleClick("John")}>Click Me </button>
```

- ❌ It will**execute immediately**during rendering, NOT on click.
- You must wrap it in a function:

```
<button onClick={(e) => handleClick(e,"John")}>Click Me </button>
```